SE4030 - Secure Software Development

ICare Security Assessment

Group security assessment report

ICare Optical Store Management System

React frontend | Express API | MongoDB | Google OpenID Connect

Prepared 25 September 2026

Contribution Table

Member details and assigned vulnerability areas are taken from the project README. Each member has three distinct findings for the review and viva.

Student ID	Student name / assigned vulnerabilities
IT22264220	**Kojithan P.Y** V01, V02, V07: Public seed route, order ownership and current administrator role; backend access-control review.
IT22172600	**Baskaran V** V03, V04, V05: Order payment/delivery permissions, server-side pricing and card-data removal; checkout regression review.
IT22223876	**Nishara T** V06, V08, V09: Prescription permissions, safe user responses and password reset; account/reset-flow review.
IT22267740	**Olivea K T** V10, V11, V12: Upload validation, ticket ownership and error disclosure; Google OIDC demo/report review.

Attribution note

The README describes these as assigned responsibilities. The repository does not establish separate code authorship for all four members. Each member should verify their actual work and retain individual evidence before submission.

Executive Summary

Submission metadata

Modified GitHub repository: TODO. YouTube demonstration (maximum 20 minutes): TODO. Semester start date: TODO. These submission details must be completed before upload.

Original project baseline: https://github.com/kojithan-y/Optical-Store-Management-System/tree/5b5b25869d92a662b1e5180eb1b4f22bcf3a7830 . This local commit is dated 11 February 2025. The original remote latest commit and semester start date still require verification for eligibility. ICare is an optical store application with React, Express, MongoDB, appointments, prescriptions, tickets, inventory, orders, and administrator workflows.

Method and scope

White-box review of the checked-in baseline source, targeted regression tests of the hardened code, and a live Google sign-in through the dedicated demo OAuth client. Findings below are supported by identifiable original routes and data flow. No external scanner result, live database exploit, penetration test, or dependency audit is claimed. Use an isolated copy of the baseline and test data for demonstrations. Baseline line references refer to commit 5b5b2586; source paths below are relative to ICare/.

Twelve distinct weaknesses were identified across destructive setup endpoints, authorization, payment integrity, privacy, reset tokens, file upload, and error handling. The highest impact issues allowed public deletion of users/products, cross-user order access, arbitrary paid status, and storage of payment card data. Code fixes are implemented. Operational work remains for historical secrets and card records.

Qualitative priority: critical - V01, V03, V05; high - V02, V04, V06, V07, V09; medium - V08, V10, V11, V12. These are review priorities based on reachable data and impact, not formal CVSS scores.

Kojithan P.Y | IT22264220

Assigned findings: V01, V02 and V07. Review focus: backend access control.

V01 Public database reset

Original: GET /api/seed in seedRoutes.js deletes all Product and User documents and repopulates default accounts without authentication.

Risk: Unauthenticated database destruction and known demo credentials.

Fix: Removed /api/seed from server.js and deleted the demo seed source.

Evidence: Baseline: backend/server.js:46 and routes/seedRoutes.js:9-12. Modified: backend/server.js.

V02 Order IDOR

Original: GET /api/orders/:id checked only for a valid bearer token and returned any order by its ID.

Risk: A customer could obtain another customer's address, items, and payment state.

Fix: Require matching order.user or a current administrator before returning an order.

Evidence: Baseline: backend/routes/orderRoutes.js:109-116. Regression: owner 200, other user 404.

V07 Stale role in JWT

Original: isAuth trusted isAdmin from a 30-day token even after a user was demoted or deleted.

Risk: Former administrators retained privileges until their token expired.

Fix: isAuth now loads the current account and role from MongoDB on every request.

Evidence: Baseline: backend/utils.js:17,21,30-47. Regression: stale admin claim yields 403 after demotion.

Baskaran V | IT22172600

Assigned findings: V03, V04 and V05. Review focus: checkout and payment regression.

V03 Unauthorized payment and delivery changes

Original: PUT /api/orders/:id/deliver and /pay required only sign-in. The card branch marked an order paid without processor confirmation.

Risk: Any user could alter fulfilment or forge a paid order.

Fix: Both routes now require administrator. The pay route records only collected Cash on Delivery payment and rejects repeated/non-COD updates. The customer Pay Now control was removed.

Evidence: Baseline: backend/routes/orderRoutes.js:123-163. Regression: customer 403, administrator 200.

V04 Client-controlled order totals

Original: POST /api/orders copied client order item prices and totals into MongoDB.

Risk: A customer could submit a lower item price or total.

Fix: Lookup catalog products by ID, validate quantity and stock, copy server-side prices and recalculate shipping, tax and total.

Evidence: Baseline: backend/routes/orderRoutes.js:26-34. Regression: client price 0.01 yields catalog price 200.

V05 Stored card data and CVV

Original: CardDetailsScreen saved PAN and CVV in localStorage; cardRoutes.js and orderModel.js stored them in MongoDB.

Risk: Sensitive payment credentials could be exposed through XSS, browser access, database breach, or API response.

Fix: Removed card form, saved-card API, card schema/model and order card fields. Checkout now supports Cash on Delivery only.

Evidence: Baseline: frontend/CardDetailsScreen.js:59-62; backend/routes/cardRoutes.js:13-22 and orderRoutes.js:36-42.

Nishara T | IT22223876

Assigned findings: V06, V08 and V09. Review focus: account and reset flows.

V06 Prescription access control

Original: Prescription list, detail, create and update routes accepted any authenticated user.

Risk: Medical prescription data could be read or modified by unrelated customers.

Fix: Applied administrator checks to prescription management routes.

Evidence: Baseline: backend/routes/prescriptionRoutes.js:33,43,57. Regression: customer 403, administrator 200.

V08 Sensitive user fields in API responses

Original: Admin user list/detail returned full Mongoose documents, including password hashes and resetToken. Update returned the full saved user.

Risk: Extra credential material could leak through a compromised admin browser or logs.

Fix: Exclude sensitive fields in list/detail queries; use a small update response object.

Evidence: Baseline: backend/routes/userRoutes.js:16,25-27,145. Modified: .select exclusions and safe update object.

V09 Unsafe password reset

Original: Reset JWT appeared in server logs, lasted three hours, was not cleared after use, and unknown emails returned 404.

Risk: Reset links could be reused or exposed; account enumeration was possible.

Fix: Generate random token, store only SHA-256 hash for 15 minutes, consume atomically, avoid logging it and return generic request response.

Evidence: Baseline: backend/routes/userRoutes.js:74-80,109-125. Modified: resetTokenExpiresAt and findOneAndUpdate.

Olivea K T | IT22267740

Assigned findings: V10, V11 and V12. Review focus: Google OIDC demo and report evidence.

V10 Unbounded file upload

Original: multer() used memory storage without a size or type limit.

Risk: An administrator upload could consume excessive memory or send unwanted file types to Cloudinary.

Fix: Limit to one 5 MB JPEG, PNG or WebP upload. Check MIME type and file signature; handle oversized uploads as 413.

Evidence: Baseline: backend/routes/uploadRoutes.js:7. Regression: a spoofed PNG MIME type is rejected.

V11 Ticket response IDOR

Original: POST /api/tickets/:id/responses allowed any authenticated user to reply to any ticket.

Risk: Private support conversations could be modified by other customers.

Fix: Allow ticket owner or current administrator only.

Evidence: Baseline: backend/routes/ticketRoutes.js:122-136. Regression: other customer 404.

V12 Internal error disclosure

Original: Global error handler returned err.message as a 500 response.

Risk: Database and implementation details could be disclosed.

Fix: Return generic 500 messages and structured 400 validation errors; log server-side.

Evidence: Baseline: backend/server.js:66. Modified: generic error handler.

Repeatable security checks

Use only an isolated test database. With owner and other-customer bearer tokens, GET /api/orders/{ownerOrderId} returns 200 for owner and 404 for the other customer. PUT /api/orders/{id}/pay and /deliver return 403 for a customer. GET /api/prescriptions returns 403 for a customer and 200 for an administrator. POST /api/tickets/{otherTicketId}/responses returns 404. These hardened outcomes are covered by an HTTP route test with mocked database models; the original outcomes follow directly from the baseline route conditions above and were not exercised against a live database.

Google OpenID Connect feature

User journey

Sign In now offers Sign in with Google. A successful first sign-in creates a non-admin ICare account bound to the Google subject (sub); later sign-ins resolve the account by sub. An existing password account with the same email is not auto-linked, preventing email-based account takeover. The app then issues its existing ICare bearer token.

Protocol and trust boundaries

The backend starts an authorization-code flow with openid email profile scopes. It sets short-lived HttpOnly SameSite=Lax state, nonce and PKCE verifier cookies. The callback compares state, exchanges the code with Google over HTTPS using the configured client secret and verifier, retrieves Google public keys, validates the RS256 ID-token signature and issuer, audience, expiry, nonce, subject and verified email, then clears flow cookies. The ICare token is delivered to the React callback via URL fragment and removed from browser history immediately.

Configuration and verification

A dedicated Google Cloud project, ICare SE4030 OAuth Demo, uses an External consent screen in Testing mode. Its Web application client, ICare Local Web Client, registers http://localhost:3000 as the browser origin and http://localhost:4000/api/auth/google/callback as the backend redirect URI. The owner account is an allowed test user. GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI and FRONTEND_ORIGIN are set only in the ignored backend .env. The frontend opens the backend authorization endpoint directly; after token verification, the backend returns to the frontend callback. Automated tests cover accepted identity and rejected signature, audience, nonce and expiration. On 25 September 2026, a live browser test reached Google consent, returned to ICare, and displayed the Google account as signed in without an administrator menu. The account owner clicked consent personally. No secret or bearer token is included in this report.

Verification and remaining work

Completed checks

Backend: npm test - 7 tests passed (catalog price tampering, stock/duplicate rejection, upload MIME spoofing, ID-token verification, stale administrator claim, HTTP owner/admin boundaries, simulated Google authorization-code flow). Frontend: npm run build completed with existing toolchain warnings. The Google preflight confirmed the backend issues a redirect to Google with state, nonce, PKCE and the configured callback. A live browser test completed Google consent and returned to an ICare session displaying the Google account. These checks do not replace a repeatable full integration suite against a test MongoDB database.

Unfixed vulnerabilities and operational risks

1. Original Git history contains backend .env. The modified working tree removes it from tracking, but old commits still contain it. Rotate MongoDB, JWT and Cloudinary credentials and consider history rewrite before publication. 2. Historical card records remain in any deployed MongoDB and backups; purge them securely under an approved retention procedure. 3. Card payments remain unavailable until a real PCI-compliant hosted gateway with server-side verification is integrated. 4. Existing bearer tokens live in localStorage and may be stolen by future XSS; migrate to a short-lived secure HttpOnly session architecture after a full frontend review. 5. A full dependency update, external scan, rate limiting and integration test suite remain future work.

Software engineering best practices

Add threat modelling for customer/staff data, an authorization matrix for every route, code review for payment and reset flows, secret scanning and .env ignore rules, regression tests for role/owner boundaries, dependency scanning, and a deployment checklist for data migrations and credential rotation.

References

OWASP Top 10: https://owasp.org/www-project-top-ten/

Google OpenID Connect: https://developers.google.com/identity/openid-connect/openid-connect

Google OAuth 2.0 web server flow: https://developers.google.com/identity/protocols/oauth2/web-server

Google ID-token verification: https://developers.google.com/identity/gsi/web/guides/verify-google-id-token